



ADVANCE DATABASE SYSTEMS

Lab Final

Group # 16

SUBMITTED BY:

Sidra Tul Muntaha (FA24-BCS-097)
Muhammad Nafay Anjum (FA24-BCS-078)

SUBMITTED TO: Dr. Basit Raza

Student Assignment Submission & Tracking System (NoSQL-Based Database Design)

1. INTRODUCTION

The Student Assignment Submission & Tracking System is designed to manage academic assignments in a structured and efficient manner. In traditional academic environments, assignment handling is often manual or distributed across multiple platforms, leading to issues such as poor tracking, data redundancy, and inconsistent grading. This system addresses these challenges through a centralized NoSQL database solution.

This report documents the complete implementation including CRUD operations, aggregation pipelines, ACID transactions, concurrency control (locking), MongoDB Views, indexing, and a fully functional GUI.

1.1 Scope

The system manages:

- Assignment creation and deadline management
- Student assignment submission with file tracking
- Automated grading and feedback storage
- Performance analytics via aggregation pipelines
- ACID transaction management
- Concurrency control via optimistic and pessimistic locking
- Read-only virtual collections via MongoDB Views

1.2 Objectives

- Design an efficient ER model and transform it to NoSQL
- Implement complete CRUD operations for all collections
- Apply aggregation pipelines for complex data analysis
- Implement ACID multi-document transactions
- Demonstrate optimistic and pessimistic concurrency control
- Build a professional, interactive GUI using Streamlit
- Optimize queries using indexing techniques

2. PROBLEM DESCRIPTION

Assignment handling in universities often relies on manual processes creating challenges:

- Data redundancy across distributed systems
- Difficulty tracking submissions in real time
- Inconsistent grading and feedback delivery
- No concurrency control when multiple instructors update records simultaneously
- Lack of analytics for performance monitoring

2.1 Why MongoDB

MongoDB was selected for several reasons:

- Flexible schema accommodates evolving data structures
- Native support for embedded documents (grade, feedback inside submissions)
- Powerful aggregation framework for multi-collection analytics
- Built-in support for ACID transactions in replica set deployments
- Horizontal scalability via sharding

2.2 Requirement Analysis

Functional Requirements

- Instructors can create and manage assignments
- Students can submit assignments online
- System stores submission details including file, timestamp, and student info
- Instructors can assign grades and provide feedback
- System generates notifications for deadlines
- System supports querying and filtering of data

Non-Functional Requirements

- Scalable to handle large datasets (thousands of records)
- Fast query processing
- Efficient storage using NoSQL structures
- Data consistency and reliability

Users of the System

- **Student** — submits assignments and views feedback

- **Instructor** — creates assignments and evaluates submissions
- **Admin** — manages system data and records

3. DATASET DETAILS AND MONGODB SCHEMA

3.1 Dataset Description

The dataset was custom-generated using a Python script (generate_data.py) to represent realistic university operations:

Collection	Documents	Description
students	1,000	Student profiles with enrolled courses
instructors	50	Instructor profiles with departments
courses	100	Course catalog with instructor links
assignments	400	Assignments with deadlines
submissions	2,000	Student submissions with grades/feedback
departments	10	Academic departments
notifications	300	System notifications

3.2 ER to NoSQL Transformation

Entities

STUDENT, INSTRUCTOR, COURSE, ASSIGNMENT, SUBMISSION, DEPARTMENT, NOTIFICATION, FEEDBACK, FILE

Mapping Rules Applied

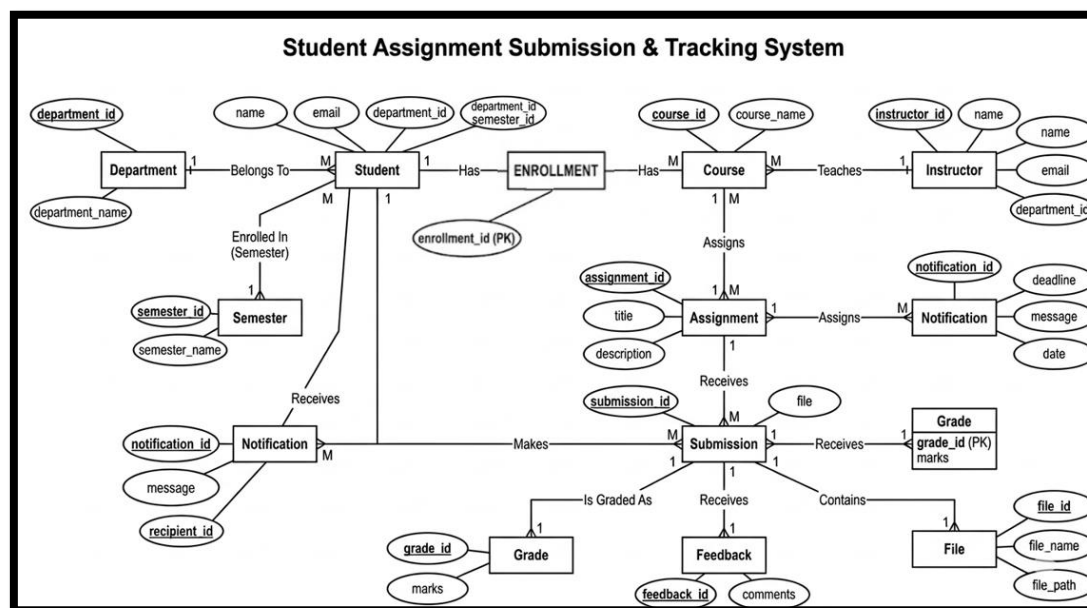
- **1:1 (Submission↔Feedback):** Feedback embedded inside submission document
- **1:M (Assignment→Submissions):** Referencing via assignment_id in submissions
- **M:N (Student↔Course):** Array of course IDs inside Student document
- **Disjoint Inheritance (Student/Instructor):** Separate collections for modularity
- **Union (Notification):** student_id can reference any student

No.	Relationship	Cardinality	Participation	Explanation
-----	--------------	-------------	---------------	-------------

1	Student – Course (via Enrollment)	M:N	Total (Student), Total (Course)	A student can enroll in multiple courses, and a course can have many students.
2	Instructor – Course	1:M	Total (Course), Partial (Instructor)	One instructor can teach multiple courses, but each course is taught by one instructor.
3	Course – Assignment	1:M	Total (Assignment), Partial (Course)	A course can have multiple assignments, but each assignment belongs to one course.
4	Student – Submission	1:M	Partial (Student), Total (Submission)	A student can submit many assignments, but each submission is made by one student.
5	Assignment – Submission	1:M	Total (Submission), Partial (Assignment)	An assignment can have many submissions, but each submission belongs to one assignment.
6	Submission – Grade	1:1	Total (Submission), Total (Grade)	Each submission has exactly one grade, and each grade is linked to one submission.
7	Submission – Feedback	1:1	Total (Submission), Total (Feedback)	Each submission has one feedback, and each feedback corresponds to one submission.

ERD

3.3 Mapping Rules Applied



- **1:1 Relationship:** Submission and Feedback follow a hybrid approach where feedback is embedded inside the submission document while still being logically treated as a separate entity in the ER model.
- **1:M Relationship:** Assignment to Submissions is implemented using referencing by storing assignment_id inside the submission documents to avoid document size limitations.
- **M:N Relationship:** Student to Course is implemented using an array of course IDs inside the Student document, reducing the need for join operations.
- **Disjoint/Overlap (Inheritance):** A separate collection approach is used for Students and Instructors to maintain modularity and clarity.
- **Union Relationship:** The recipient_id field in the Notification collection can reference either a Student or an Instructor, demonstrating a union-type relationship.

3.4 Final MongoDB Collections

1. students Collection

Purpose:

Stores all student-related information including personal details, academic data, and enrolled courses.

Key Features:

- Contains student-specific attributes such as name, email, department, and semester
- Stores enrolled courses as an array to represent M:N relationship
- Links to department and semester using references

Reason:

- Efficient representation of student data
- Reduces need for join operations using embedded course arrays
- Supports scalability for large student datasets

2. instructors Collection

Purpose:

Stores information about instructors responsible for teaching courses.

Key Features:

- Contains instructor details such as name, email, and department
- References department for organizational structure

Reason:

- Separates instructor data for better modularity
- Enables easy mapping with courses

3. courses Collection**Purpose:**

Stores all course-related information offered in the system.

Key Features:

- Contains course name and instructor reference
- Establishes relationship between instructor and course

Reason:

- Supports efficient course management
- Enables linking with assignments and students

4. assignments Collection**Purpose:**

Stores all assignments created for different courses.

Key Features:

- Contains assignment details such as title, deadline, and course reference
- Linked with courses using referencing

Reason:

- Allows efficient retrieval of assignments per course
- Keeps assignment data lightweight and scalable

5. departments Collection**Purpose:**

Stores information about different academic departments.

Key Features:

- Contains department name and unique identifier
- Referenced by students and instructors

Reason:

- Avoids redundancy by centralizing department data
- Maintains consistency across records

6. notifications Collection

Purpose:

Stores system-generated notifications related to assignments and updates.

Key Features:

- Contains message, date, and student reference
- Supports communication of important events

Reason:

- Enables tracking of system alerts
- Improves user engagement and awareness

7. submissions Collection

Purpose:

Stores all assignment submissions made by students.

Key Features:

- Contains student and assignment references
- Embeds file, grade, and feedback data within the document
- Stores submission timestamps

Reason:

- Embedding reduces query complexity and improves performance
- All related data (file, grade, feedback) is accessed in a single query
- Optimized for high-frequency read/write operations

CONCLUSION

- Semester is stored as a simple attribute and not as a separate collection
- The above collection design ensures efficient data storage, scalability, and optimized query performance by combining both embedding and referencing strategies.

3.5 NoSQL Database Design (JSON Files)

1- Students Collection

```
students.json X instructors.json courses.json assignments.json departments.json notifications.j
students.json > ...
1  [
2    {
3      "_id": 0,
4      "name": "Student_0",
5      "email": "student0@gmail.com",
6      "department_id": 3,
7      "semester_id": 2,
8      "courses": [
9        28,
10       3
11     ]
12   },
13   {
14     "_id": 1,
15     "name": "Student_1",
16     "email": "student1@gmail.com",
17     "department_id": 0,
18     "semester_id": 5,
19     "courses": [
20       66,
21       42
22     ]
23   },
24   {
25     "_id": 2,
26     "name": "Student_2",
27     "email": "student2@gmail.com",
28     "department_id": 0,
29     "semester_id": 7,
30     "courses": [
31       27,
32       25
33     ]
34   }
35 ]
```

2- Instructor Collection

```
instructors.json > ...
1  [
2    {
3      "_id": 0,
4      "name": "Instructor_0",
5      "email": "instructor0@uni.edu",
6      "department_id": 7
7    },
8    {
9      "_id": 1,
10     "name": "Instructor_1",
11     "email": "instructor1@uni.edu",
12     "department_id": 9
13   },
14   {
15     "_id": 2,
16     "name": "Instructor_2",
17     "email": "instructor2@uni.edu",
18     "department_id": 5
19   },
20   {
21     "_id": 3,
22     "name": "Instructor_3",
23     "email": "instructor3@uni.edu",
24     "department_id": 4
25   },
26   {
27     "_id": 4,
28     "name": "Instructor_4",
29     "email": "instructor4@uni.edu",
30     "department_id": 2
31   }
32 ]
```

3- Courses Collection

```
students.json | instructors.json | courses.json | assignments.json | departments.json | notifications.json
```

```
courses.json > ...
1  [
2    {
3      "_id": 0,
4      "course_name": "Course_0",
5      "instructor_id": 44
6    },
7    {
8      "_id": 1,
9      "course_name": "Course_1",
10     "instructor_id": 45
11   },
12   {
13     "_id": 2,
14     "course_name": "Course_2",
15     "instructor_id": 41
16   },
17   {
18     "_id": 3,
19     "course_name": "Course_3",
20     "instructor_id": 14
21   },
22   {
23     "_id": 4,
24     "course_name": "Course_4",
25     "instructor_id": 22
26   },
27   {
28     "_id": 5,
29     "course_name": "Course_5",
30     "instructor_id": 26
31   },
32   {
```

4- Assignments Collection

```
{} students.json {} instructors.json {} courses.json X {} assignments.json X {} departments.json {} notifications.j ▶ {} ...

{} assignments.json > ...
1  [
2    {
3      "_id": 0,
4      "title": "Assignment_0",
5      "course_id": 84,
6      "deadline": "2026-04-20"
7    },
8    {
9      "_id": 1,
10     "title": "Assignment_1",
11     "course_id": 17,
12     "deadline": "2026-04-2"
13   },
14   {
15     "_id": 2,
16     "title": "Assignment_2",
17     "course_id": 27,
18     "deadline": "2026-04-20"
19   },
20   {
21     "_id": 3,
22     "title": "Assignment_3",
23     "course_id": 84,
24     "deadline": "2026-04-10"
25   },
26   {
27     "_id": 4,
28     "title": "Assignment_4",
29     "course_id": 43,
30     "deadline": "2026-04-10"
31   },
32   {
```

5- Departments Collection

```
{ } departments.json > ...
1  [
2    {
3      "_id": 0,
4      "department_name": "Department_0"
5    },
6    {
7      "_id": 1,
8      "department_name": "Department_1"
9    },
10   {
11     "_id": 2,
12     "department_name": "Department_2"
13   },
14   {
15     "_id": 3,
16     "department_name": "Department_3"
17   },
18   {
19     "_id": 4,
20     "department_name": "Department_4"
21   },
22   {
23     "_id": 5,
24     "department_name": "Department_5"
25   },
26   {
27     "_id": 6,
28     "department_name": "Department_6"
29   },
30   {
31     "_id": 7,
32     "department_name": "Department_7"
33   }
34 ]
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} JSON Go Live Background Prettier

6- Notifications Collection

```
notifications.json > ...
1  [
2    {
3      "_id": 0,
4      "message": "Grades released",
5      "date": "2026-04-1",
6      "student_id": 990
7    },
8    {
9      "_id": 1,
10     "message": "Grades released",
11     "date": "2026-04-26",
12     "student_id": 838
13   },
14   {
15     "_id": 2,
16     "message": "Assignment deadline approaching",
17     "date": "2026-04-8",
18     "student_id": 196
19   },
20   {
21     "_id": 3,
22     "message": "Grades released",
23     "date": "2026-04-27",
24     "student_id": 281
25   },
26   {
27     "_id": 4,
28     "message": "Assignment deadline approaching",
29     "date": "2026-04-13",
30     "student_id": 733
31   },
32   {
33     "_id": 5,
34     "message": "Grades released",
35     "date": "2026-04-13",
36     "student_id": 733
37   }
38 ]
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} JSON Go Live Background Prettier

7- Submission Collection

submissions.json submission.py X

submission.py

```
1  [
2    {
3      "_id": 0,
4      "student_id": 694,
5      "assignment_id": 88,
6      "submission_date": "2026-04-25",
7      "file": {
8        "file_name": "file_0.pdf",
9        "file_path": "/uploads/file_0.pdf"
10     },
11     "grade": {
12       "marks": 45
13     },
14     "feedback": {
15       "comments": "Excellent"
16     }
17   },
18   {
19     "_id": 1,
20     "student_id": 11,
21     "assignment_id": 327,
22     "submission_date": "2026-04-26",
23     "file": {
24       "file_name": "file_1.pdf",
25       "file_path": "/uploads/file_1.pdf"
26     },
27     "grade": {
28       "marks": 41
29     },
30     "feedback": {
31       "comments": "Average"
32     }
33   }
34 ]
```

ve Share Ln 32002, Col 2 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live Background

4. PROJECT IMPLEMENTATION

4.1 Technology Stack

Component	Technology	Purpose
Database	MongoDB (Local/Atlas)	Primary data store
Backend	Python 3 + PyMongo	Database interaction
Frontend	Streamlit	Interactive dashboard
Visualization	Pandas, Streamlit Charts	Bar charts, data tables

4.2 CRUD Operations

CRUD operations were implemented for all 7 collections via the GUI with three tabs per collection: View/Delete, Add, Edit.

Create (Insert Operations)

Data was inserted into collections using both manual insertion and bulk JSON import.

Example:

The screenshot shows the 'Students Collection' form in the SASTS application. The form has fields for 'name', 'email', 'department_id', and 'semester_id'. The 'name' field contains 'Nafee', 'email' contains 'abc@gmail.com', 'department_id' is set to '1', and 'semester_id' is set to '4'. There is a 'courses' field with a placeholder '0,1,2' and a 'Save Document' button at the bottom.

Read (Retrieve Operations)

Read operations were used to fetch data from collections.

Example:

The screenshot shows the 'Students Collection' table view in the SASTS application. A green banner at the top indicates 'Inserted document_id=1022'. Below the table, there is a 'Total documents: 1017' and a 'Remove a document' button.

_id	name	email	department_id	semester_id	courses
1006	1011 Nafay	abc@gmail.com		3	4 [7,8,9]
1007	1012 Nafay	abc@gmail.com		3	4 [7,8,9]
1008	1013 Nafay	abc@gmail.com		3	4 [7,8,9]
1009	1014 Nafay	abc@gmail.com		3	4 [7,8,9]
1010	1015 Nafay	abc@gmail.com		3	5 [7,8,9]
1011	1016 Nafay	abc@gmail.com		3	5 [7,8,9]
1012	1017 Test Student	test@gmail.com		1	3 [0,110,20]
1013	1018 Test Student	test@gmail.com		1	3 [0,110,20]
1014	1019 Test Student 123	test123@gmail.com		0	0 [0,1]
1015	1021 sidra	abc@email.com		7	4 [9,17]
1016	1022 Nafee	abc@gmail.com		1	4 [0,13]

Update (Modify Operations)

Update operations were performed to modify an entry.

Example :

Students Collection

Browse & Delete

Insert New

Update

Enter _id to load

1022

Load Document

Editing document _id=1022

name

Nafee

email

abc@gmail.com

department_id

1

semester_id

4

courses

0,13,17

Press Enter to submit form

Save Changes

Delete (Remove Operations)

Delete operations were used to remove unnecessary or outdated records.

Example:

Students

Students Collection

Deleted document _id=1022

Browse & Delete

Insert New

Update

_id	name	email	department_id	semester_id	courses
6	Student_6	student6@gmail.com		7	[22, 36]
7	Student_7	student7@gmail.com		2	[20, 51]
8	Student_8	student8@gmail.com		6	[99, 28]
9	Student_9	student9@gmail.com		6	[59, 66]
10	Student_10	student10@gmail.com		6	[40, 87]
11	Student_11	student11@gmail.com		4	[70, 54]
12	Student_12	student12@gmail.com		8	[66, 80]
13	Student_13	student13@gmail.com		9	[50, 38]
14	Student_14	student14@gmail.com		2	[73, 30]
15	Student_15	student15@gmail.com		1	[76, 99]
16	Student_16	student16@gmail.com		5	[6, 58]

Total documents: 1016

Remove a document

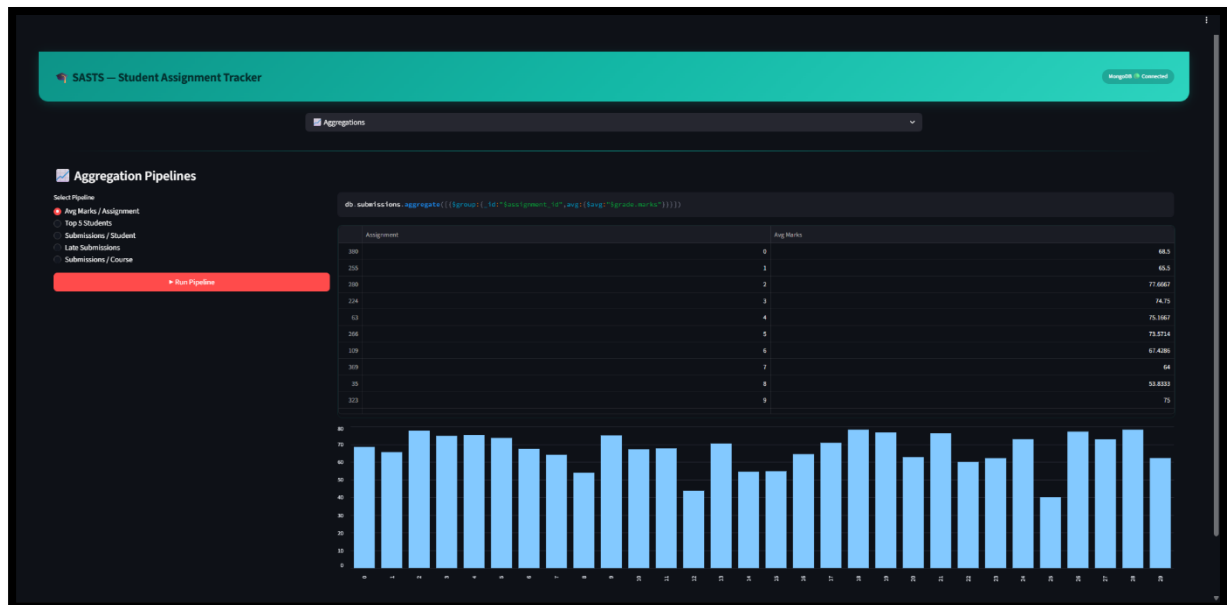
4.3 Aggregation Pipelines

Pipeline 1: Average Marks per Assignment

QUERY:

```
db.submissions.aggregate([
  { $group: { _id: "$assignment_id",
    averageMarks: { $avg: "$grade.marks" } } }
])
```

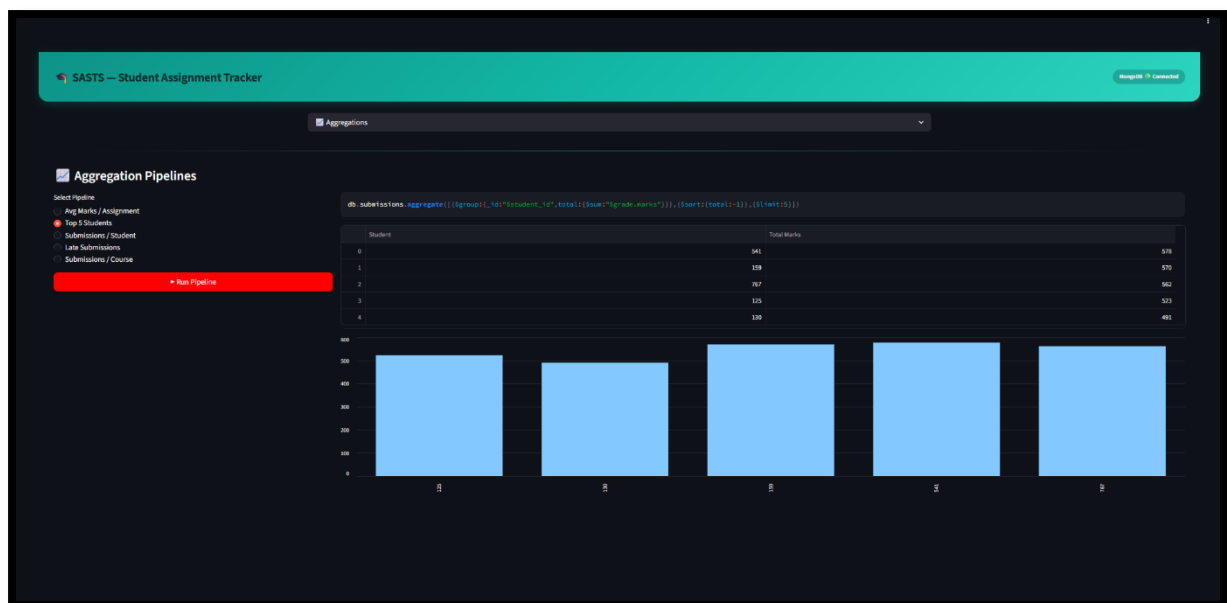
Calculates average performance per assignment. Helps evaluate difficulty.



Pipeline 2: Top 5 Performing Students

QUERY:

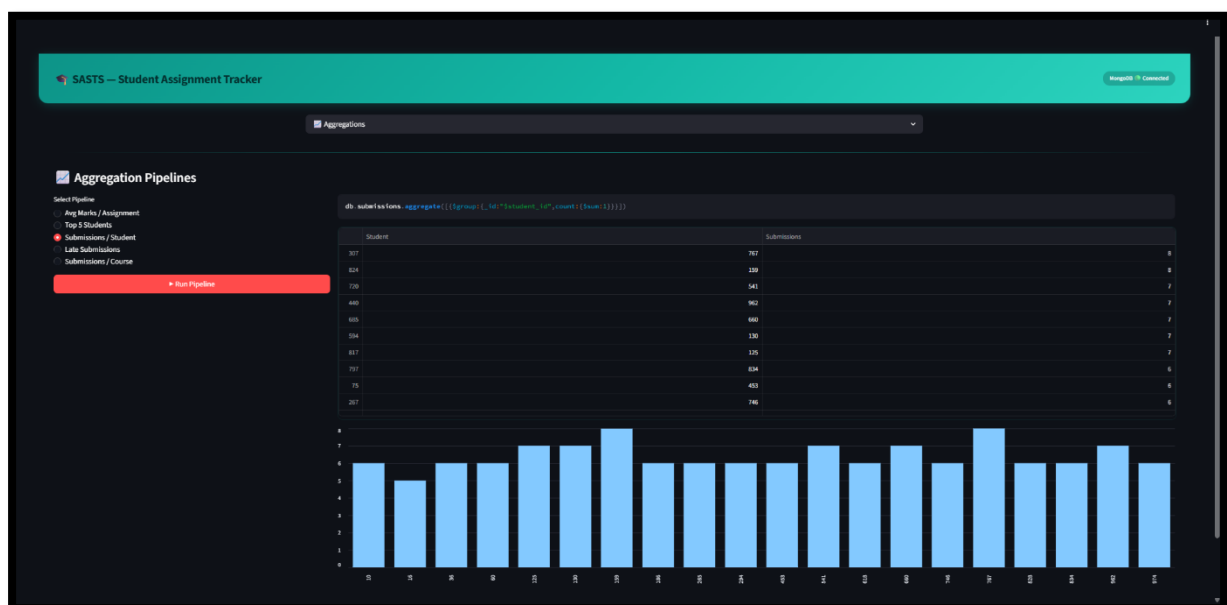
```
db.submissions.aggregate([
  { $group: { _id: "$student_id",
    totalMarks: { $sum: "$grade.marks" } } },
  { $sort: { totalMarks: -1 } },
  { $limit: 5 }
])
```



Pipeline 3: Total Submissions per Student

QUERY:

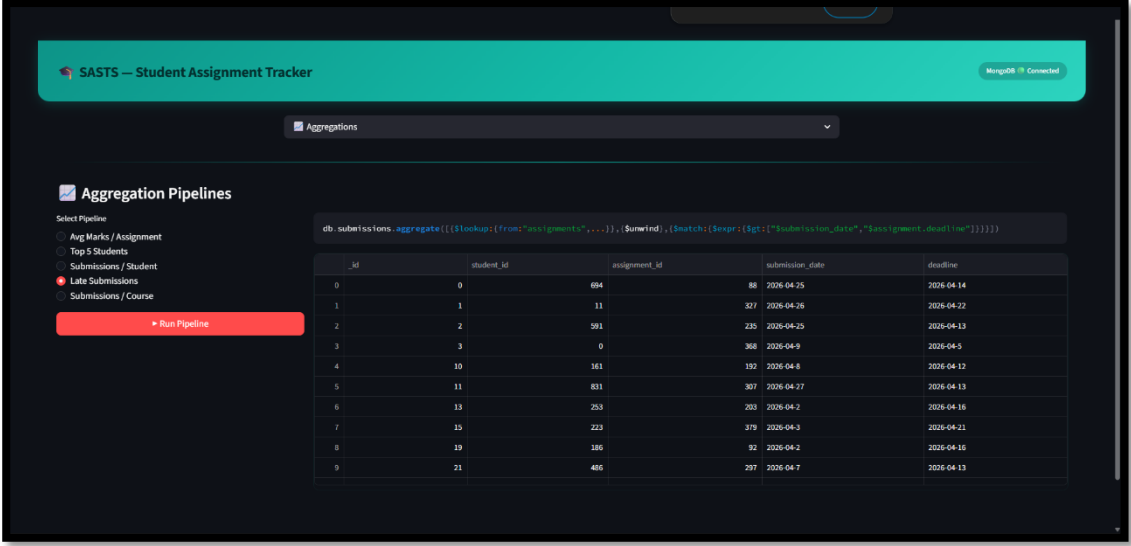
```
db.submissions.aggregate([
  { $group: { _id: "$student_id",
    totalSubmissions: { $sum: 1 } } }
])
```



Pipeline 4: Late Submissions Detection

QUERY:

```
db.submissions.aggregate([
  { $lookup: { from: "assignments", localField: "assignment_id",
    foreignField: "_id", as: "assignment" } },
  { $unwind: "$assignment" },
  { $match: { $expr: { $gt: ["$submission_date",
    "$assignment.deadline"] } } }
])
```



SASTS — Student Assignment Tracker

MongoDB Connected

Aggregations

Aggregation Pipelines

Select Pipeline

- Avg Marks / Assignment
- Top 5 Students
- Submissions / Student
- Late Submissions
- Submissions / Course

Run Pipeline

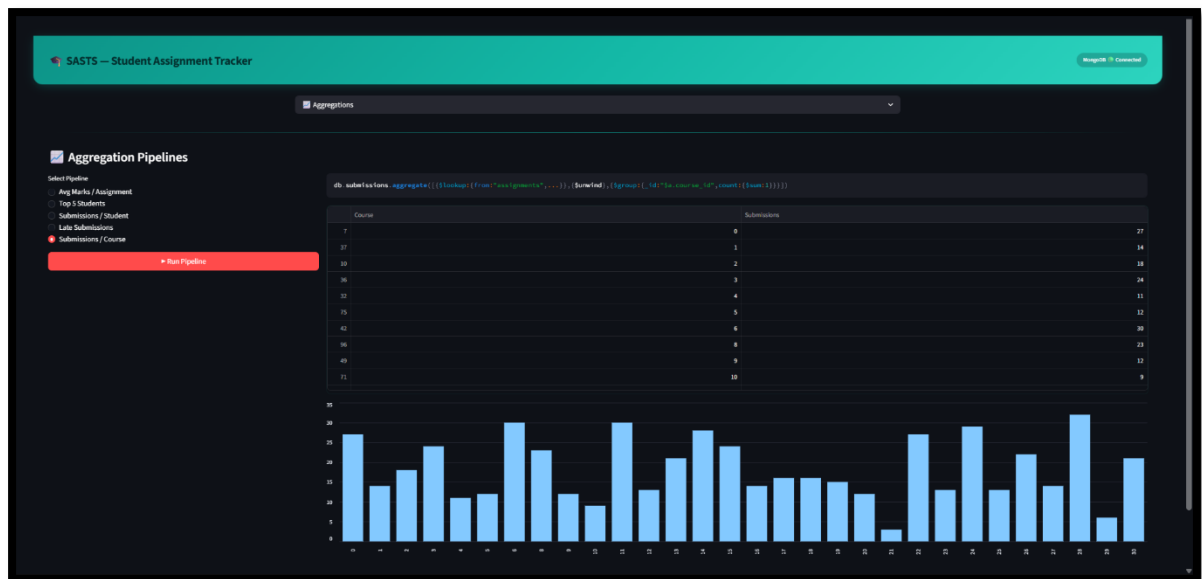
```
db.submissions.aggregate([{$lookup:{from:"assignments",...}},{$unwind},{match:{$expr:{$gt:["$submission_date","$assignment.deadline"]}}])
```

_id	student_id	assignment_id	submission_date	deadline
0	0	694	88 2026-04-25	2026-04-14
1	1	11	327 2026-04-26	2026-04-22
2	2	583	235 2026-04-25	2026-04-13
3	3	0	368 2026-04-9	2026-04-5
4	10	161	192 2026-04-8	2026-04-12
5	11	831	307 2026-04-27	2026-04-13
6	13	253	203 2026-04-2	2026-04-16
7	15	223	379 2026-04-3	2026-04-21
8	19	186	92 2026-04-2	2026-04-16
9	21	466	297 2026-04-7	2026-04-13

Pipeline 5: Submissions per Course

QUERY:

```
db.submissions.aggregate([
  { $lookup: { from: "assignments", localField: "assignment_id",
    foreignField: "_id", as: "assignment" } },
  { $unwind: "$assignment" },
  { $group: { _id: "$assignment.course_id",
    totalSubmissions: { $sum: 1 } } }
])
```



4.4 ACID Transactions

Theory

- **Atomicity:** All operations succeed or all fail together
- **Consistency:** Database moves from one valid state to another
- **Isolation:** Concurrent transactions don't interfere
- **Durability:** Committed data persists after failure

Submit Assignment Transaction

This transaction performs three atomic steps:

1. Verify student exists in students collection
2. Verify assignment exists in assignments collection
3. Insert new submission document

If any step fails, the entire transaction is aborted and rolled back.

The screenshot shows the 'ACID Transactions' interface. At the top, there's a 'Transactions' dropdown menu. Below it, four ACID properties are listed: A (Atomicity: All or nothing), C (Consistency: Valid state transitions), I (Isolation: No interference), and D (Durability: Persists after failure). The 'Submit Assignment Transaction' section contains input fields for Student ID (5), Assignment ID (10), and Marks (85). A checkbox for 'Force failure (simulate rollback)' is unchecked. A red 'Execute Transaction' button is at the bottom left. On the right, a list of steps shows the transaction progress: Step 1: Student 'Student_5' found, Step 2: Assignment 'Assignment_10' found, Step 3: Submission #2002 inserted, and Step 4: Transaction COMMITTED.

This screenshot shows the same 'ACID Transactions' interface, but with a different outcome. The 'Force failure (simulate rollback)' checkbox is now checked. The 'Execute Transaction' button is still present. The steps on the right are: Step 1: Student 'Student_5' found, Step 2: Assignment 'Assignment_10' found, Step 3: Δ FORCED FAILURE, and Step 4: ROLLBACK: Forced failure for rollback demo.

4.5 CONCURRENCY CONTROL AND LOCKING

4.5.1 Optimistic Locking

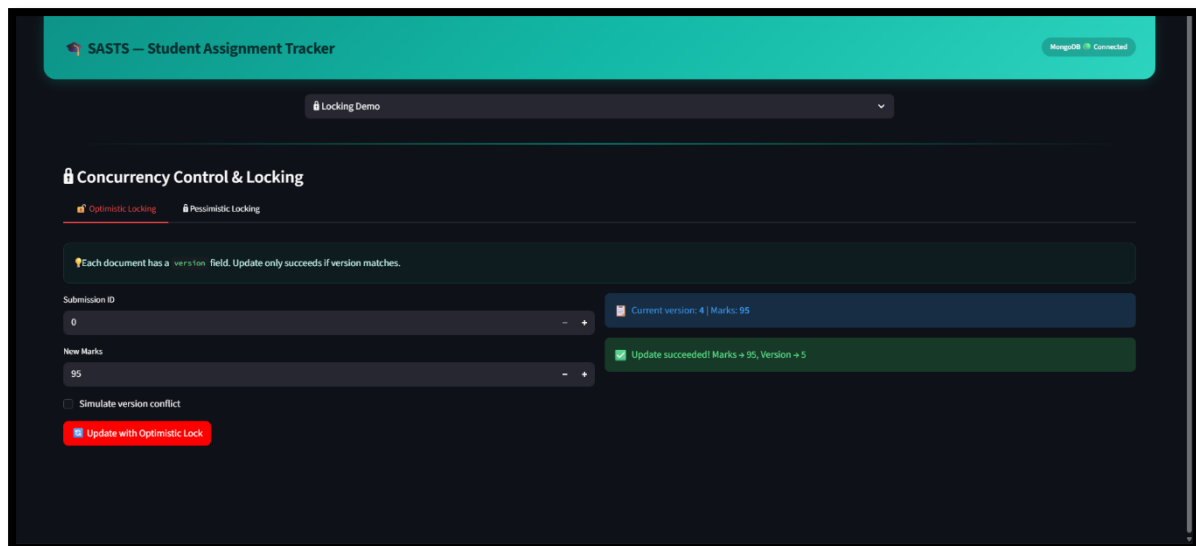
Each document carries a version field. Update only succeeds if version matches:

QUERY:

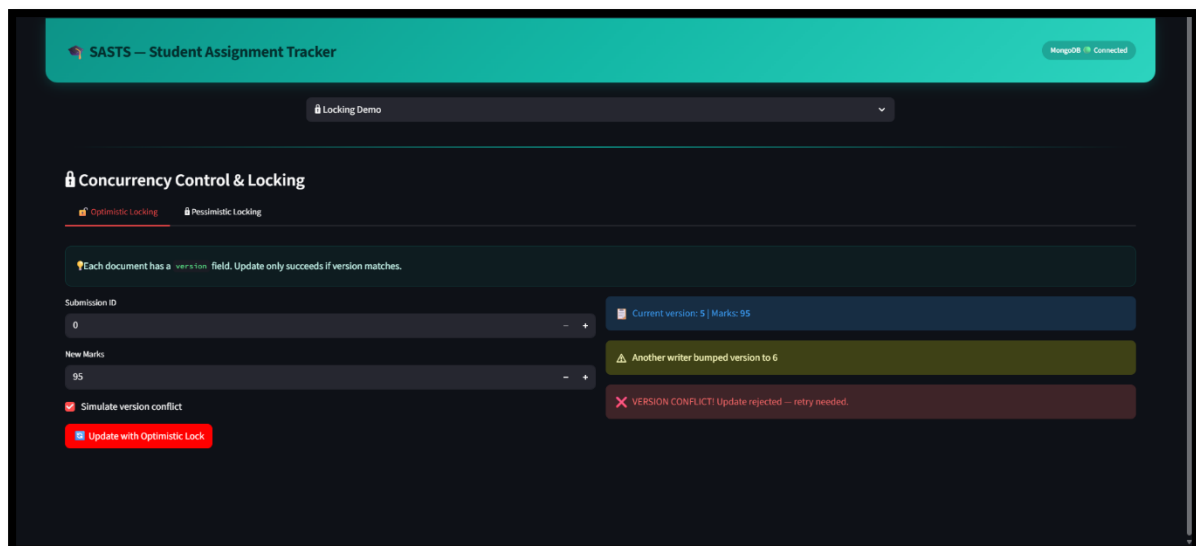
```
result = db.submissions.updateOne(
  { _id: 0, version: currentVersion },
  { $set: { "grade.marks": 95, version: currentVersion + 1 } }
)
```

// If matchedCount == 0 → VERSION CONFLICT, retry needed

Optimistic lock success:



Optimistic lock conflict:

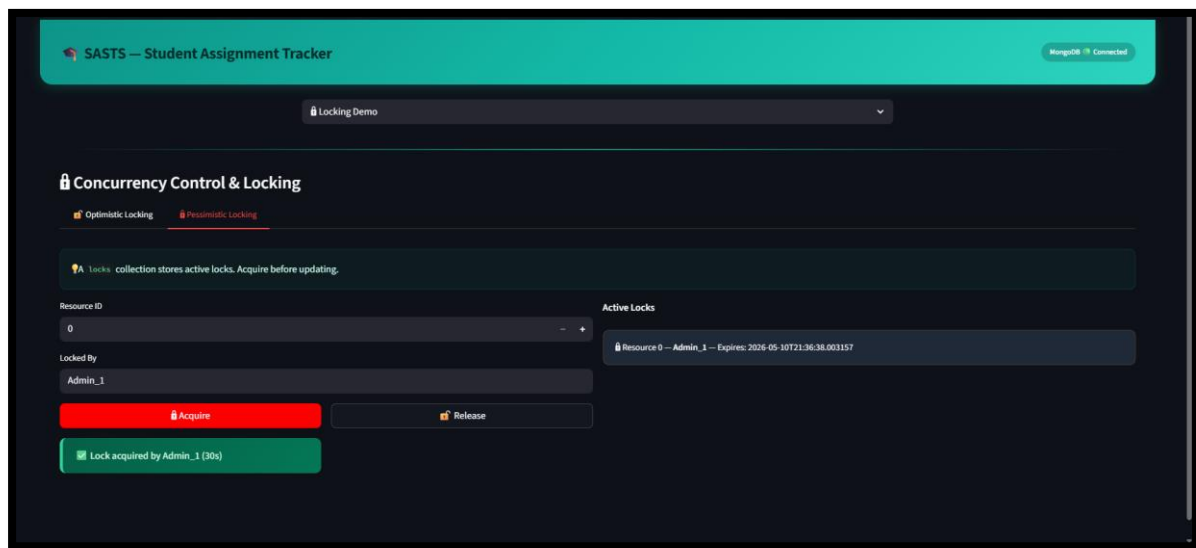


4.5.2 Pessimistic Locking

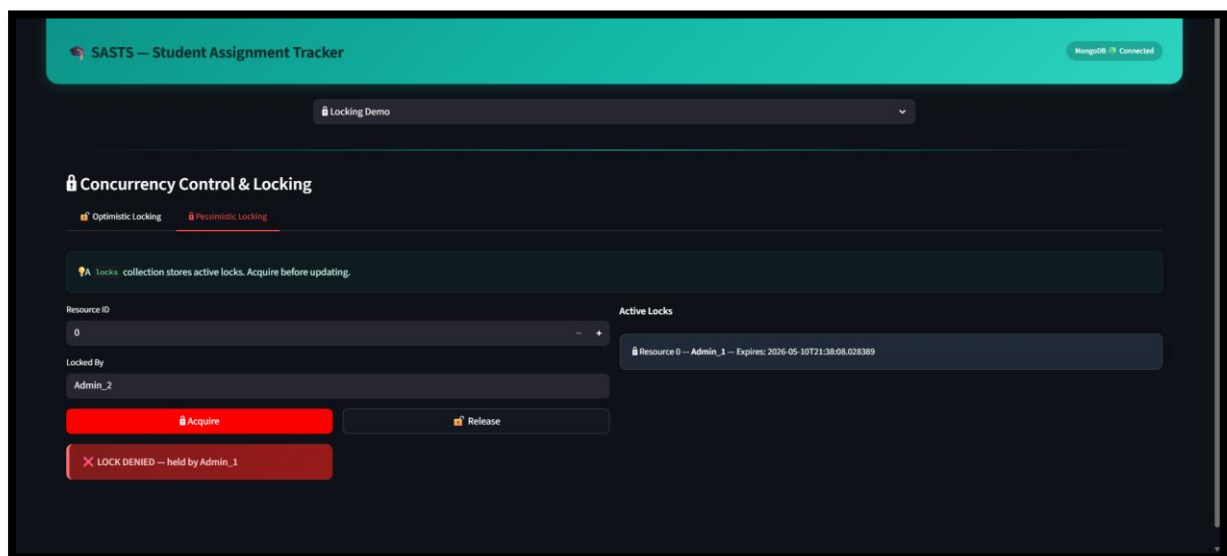
A locks collection stores active locks with expiry:

- Admin acquires lock → lock document inserted (30s expiry)
- Another admin tries → LOCK DENIED
- Lock released or expires → resource available again

Lock Acquired by Admin_1:



Lock denied for Admin_2:

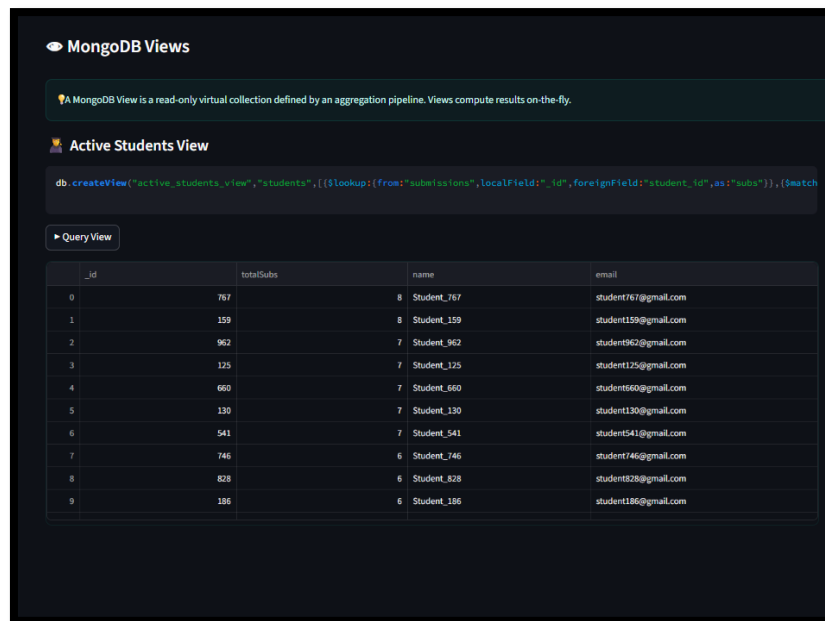


4.6 MongoDB Views

A View is a read-only virtual collection defined by an aggregation pipeline.

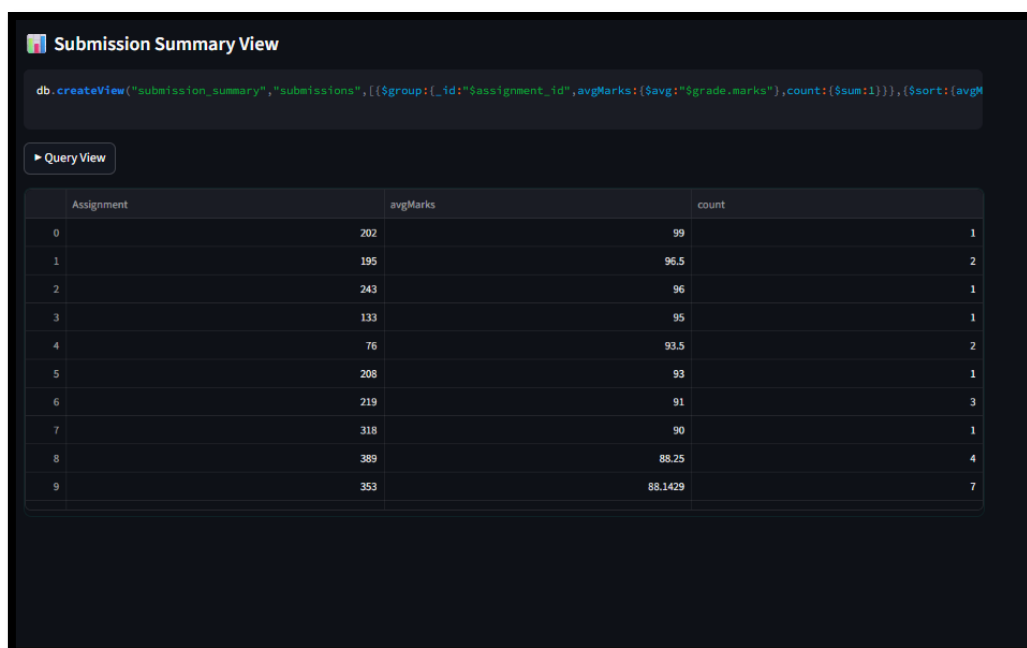
Active Students View

```
db.createView("active_students_view", "students", [  
  { $lookup: { from: "submissions", localField: "_id",  
    foreignField: "student_id", as: "subs" } },  
  { $match: { "subs.0": { $exists: true } } },  
  { $project: { name: 1, email: 1, totalSubs: { $size: "$subs" } } } ])
```



Submission Summary View

```
db.createView("submission_summary_view", "submissions", [
  { $group: { _id: "$assignment_id",
    avgMarks: { $avg: "$grade.marks" },
    count: { $sum: 1 } } },
  { $sort: { avgMarks: -1 } }
])
```



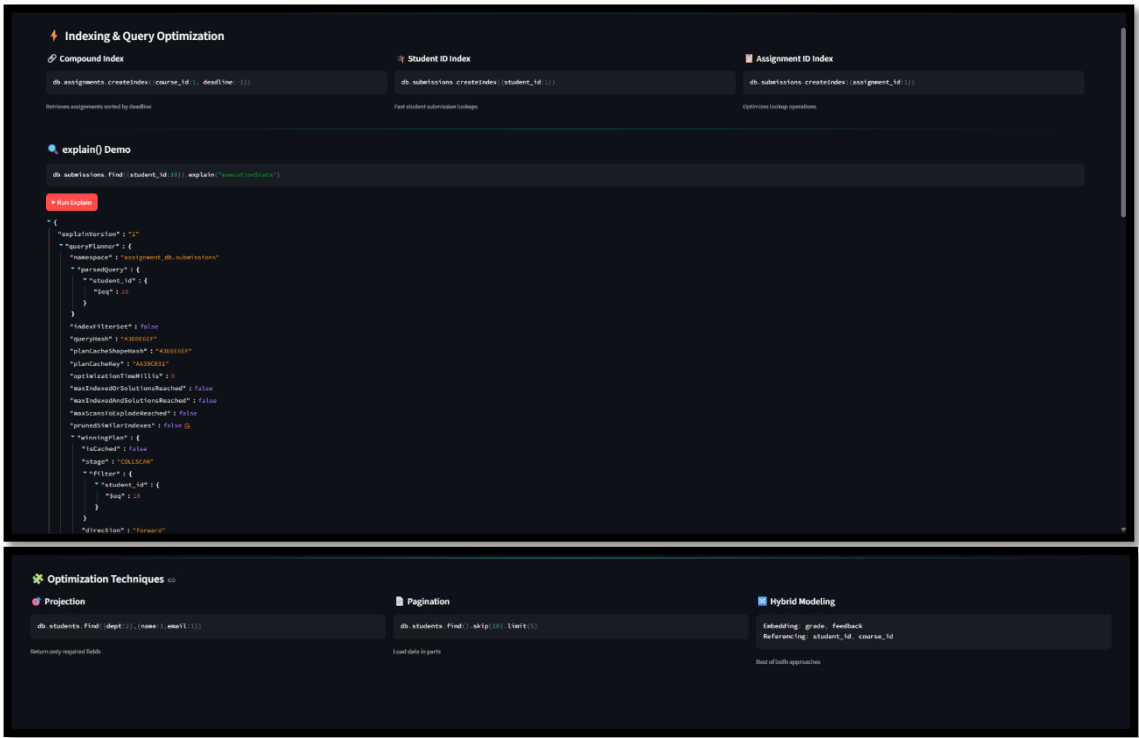
5. INDEXING AND QUERY OPTIMIZATION

5.1 Indexes Implemented

Index	Command	Purpose
Compound	db.assignments.createIndex({course_id:1, deadline:-1})	Retrieve + sort assignments by course
Student ID	db.submissions.createIndex({student_id:1})	Fast student submission lookups
Assignment ID	db.submissions.createIndex({assignment_id:1})	Optimize lookup operations

5.2 Query Optimization Techniques

- explain():** Verify index usage and measure execution stats
- Projection:** Return only required fields — db.students.find({dept:2}, {name:1, email:1})
- Pagination:** db.students.find().skip(10).limit(5)
- Hybrid Data Modeling:** Embedding (grade, feedback) for fast reads; Referencing (student_id, course_id) for scalability



6. SHARDING AND SCALABILITY

6.1 Sharding

Sharding is MongoDB's horizontal scaling mechanism distributing data across multiple shards.

Components:

- **Shard Servers** (store data portions), **Config Servers** (metadata), **mongos Router** (query routing)

6.2 Shard Key Selection

Collection	Shard Key	Strategy	Reason
students	{_id: "hashed"}	Hashed	Even distribution
submissions	{student_id:1, assignment_id:1}	Ranged Compound	Common query pattern
assignments	{course_id: 1}	Ranged	Course-based queries

6.3 Implementation

javascript

```
sh.enableSharding('assignment_db')
```

```
sh.shardCollection('assignment_db.students', { _id: 'hashed' })
```

```
sh.shardCollection('assignment_db.submissions', { student_id: 1, assignment_id: 1 })
```

6.4 Scalability Benefits

- **Horizontal Scaling:** Add shards without downtime
- **Query Parallelism:** mongos routes to relevant shards only
- **Replica Sets per Shard:** High availability with automatic failover

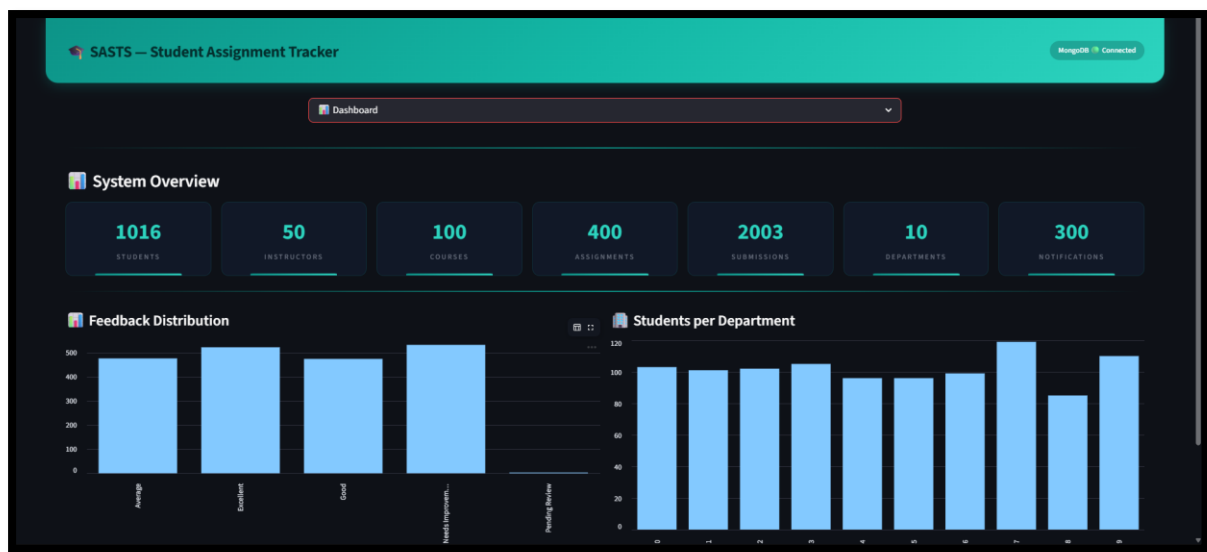
7. GRAPHICAL USER INTERFACE (GUI)

Built using **Streamlit** with dark theme.

13 pages accessible:

Module	Features
Dashboard	Live collection counts, grade distribution chart, department-wise analytics chart
Students	Add student records, edit student details, view student information, delete student records
Instructors	Add instructors, update instructor profiles, view instructor list, remove instructors
Courses	Create courses, edit course information, view available courses, delete courses
Assignments	Add assignments, update assignment details, view assignment list, delete assignments
Submissions	View submissions, track submission status, delete submission records
Departments	Add departments, edit department information, view department data, delete departments
Notifications	Create notifications, edit notifications, view alerts and updates, delete notifications
Aggregations	5 MongoDB aggregation pipelines, pipeline code display, data visualization charts
Transactions	ACID transaction demonstration, step-by-step execution logging
Locking Demo	Optimistic locking implementation, pessimistic locking implementation
Views	Active Students virtual collection, Submission Summary virtual collection
Indexing	Index information cards, explain() query analysis demo, query optimization techniques

Dashboard GUI:



8. TESTING

8.1 CRUD Testing

- Insert with auto-incremented `_id` — verified unique IDs
- Read with pagination — verified data table display
- Update by load-then-edit — confirmed modifications saved
- Delete non-existent ID — shows error correctly

8.2 Transaction Testing

- Normal flow: All 3 steps succeed → submission inserted
- Forced failure: Steps 1-2 pass → Step 3 fails → rollback confirmed
- Invalid student ID → immediate failure

8.3 Locking Testing

- Optimistic success: version matches → update applied
- Optimistic conflict: simulated → update rejected
- Pessimistic acquire/deny/release: all working correctly

9. CHALLENGES AND SOLUTIONS

Challenge	Solution
Large datasets (2000+ submissions)	Pagination with <code>.limit()</code> and <code>.skip()</code>
Cross-collection queries	<code>\$lookup</code> in aggregation pipelines
Concurrent data corruption	Optimistic + Pessimistic locking
Nested documents in tables	Flattened to string for display
Real-time GUI updates	Streamlit <code>st.rerun()</code> after modifications

10. CONCLUSION

The Student Assignment Submission & Tracking System was successfully developed using MongoDB. The project demonstrates:

- **NoSQL Design** with hybrid embedding/referencing strategy
- **Complete CRUD** for all 7 collections through interactive GUI
- **5 Aggregation Pipelines** with live chart visualization
- **ACID Transactions** with validation and rollback
- **Concurrency Control** via optimistic and pessimistic locking
- **MongoDB Views** for reusable read-only virtual collections
- **Indexing & Optimization** reducing query time from $O(n) \rightarrow O(\log n)$
- **Sharding Strategy** providing horizontal scalability path
- **Professional GUI** built with Streamlit, Pandas, and PyMongo

The system demonstrates that MongoDB is a fully-featured database platform capable of supporting enterprise-grade features including transactions, concurrency control, virtual views, and horizontal scalability.